

The mutation table

Scenario → mutation applied → the assertion that catches it.

Regenerate with:

```
.\gradlew.bat mutationReport
```

56 mutations, 56 caught. Every row below was executed: the broken implementation is a real class in `src/test/java/ise/mutants/`, and `ise.drill.MutationDrillTest` asserts that the corresponding test class goes red against it. Nothing here is a claim — if a row stopped being true, `gradlew test` would fail.

The last column is the one to study. It is not "the test failed"; it is **which single assertion did the work**, and therefore which line you cannot afford to leave out.

Mocking

Scenario	Mutation applied	Caught by	Why nothing else catches it
s01_setup_annotations	Collaborator is consulted, answer ignored — entry always granted	<code>invalidMembershipDeniesEntry</code> → <code>assertFalse(granted)</code>	<code>verify()</code> passes : the mock really was called. Only the return value notices the answer was thrown away.
s02_setup_manual	Terminal result ignored — a declined card reports success	<code>declinedCardIsNotSettled</code> → <code>assertFalse(settled)</code>	Same shape. The interaction is correct; the decision is not.
s03_setup_support	Label printed before the stock check — a collaborator is called that should be skipped	<code>unreservableOrderIsNotPrinted</code> → the unrecorded printer mock	The return value is <code>null</code> either way, so <code>assertNull</code> alone proves nothing.
s05_subbing	Sequence consulted once, the value reused for every ticket	<code>consecutiveReturns</code> → <code>verify()</code> and <code>assertIterableEquals</code>	Two unconsumed expectations <i>and</i> three identical codes. A size check passes.
s06_voidmethods	Loop body deleted — only the flush survives	<code>allReadingsAreRecordedThenFlushed</code> → <code>verify(log)</code>	<code>runCycle</code> returns void and the SUT has no state. <code>verify()</code> is the only witness that exists.
s07_callcounts	Off-by-one in the retry loop (2 attempts instead of 3)	<code>allAttemptsAreExhausted</code> → <code>.times(3)</code> + <code>verify()</code>	The return value is <code>false</code> in both versions. The count is the behaviour.
s08_exceptions	Catch block emptied — the failure counter is not incremented	<code>failureIsHandledGracefully</code> → <code>assertEquals(1, getFailedAttempts())</code>	<code>verify()</code> passes and <code>assertFalse</code> passes. Only the state assertion sees it.
s10_capture	Quantity dropped when building the invoice (10.00 instead of 30.00)	<code>singleCapture</code> → <code>assertEquals(30.0, captured.getValue().getTotal())</code>	<code>save()</code> is called exactly once either way, so <code>verify()</code> is blind.
s13_multiplemocks	Courier booked without a reservation	<code>unavailableStockIsNotDispatched</code> → the unrecorded courier mock	Returns <code>null</code> in both versions. <code>assertNull</code> proves nothing at all here.

Scenario	Mutation applied	Caught by	Why nothing else catches it
exam_vehicle	assign() called despite a failed engine check	testAssignWithEngineFailure → the unrecorded assign on a default mock	⚠ The driver list is empty either way, because this mutant's assign() returns false. State assertions alone score zero on this task. This is the exact defect the graded feedback named.
exam_vehicle	assign() skipped entirely	testAssignDriverFailure → verify(vehicleMock)	⚠ Both state assertions pass against this mutant — an empty driver list is what "refused" and "never asked" have in common. Leaving verify() out is precisely how this test scored zero.
exam_vehicle	Driver never told about the vehicle	testAssignDriverSuccessful → assertEquals(1, driver.getAssignedVehicles().size())	The mirror image: verify() passes here, both recorded calls happened. This is why you always need both halves.
exam_docking	dock() never executed (body deleted)	testDockAstronautFailure → verify(spaceshipMock)	⚠ An astronaut with no ships is exactly what an empty body produces. The graded feedback said this literally: "make sure ... fails if the dock() method is never executed".

Testing

Scenario	Mutation applied	Caught by	Why nothing else catches it
s02_aaa	deposit() body deleted	depositIncreasesTheBalance → assertEquals(150.0, getBalance())	—
s02_aaa	Guard moved after the state change	negativeDepositIsRejected → the assertEquals(100.0) after the assertThrows	The assertThrows alone passes: the exception is still thrown, just too late.
s03_assertions	admit() does nothing, occupancyRate() returns a constant	booleans → assertFalse(isEmpty()); arraysAndIterables	The assertTrue(isEmpty()) half passes. The negative half is what breaks.
s04_exceptions	Validation guard removed from the setter	setterThrows, messageIsAsserted, tryCatchMakesATestUseless	The try/catch test only catches it because of its trailing assertEquals — which is the lesson.
s05_assertional	Item stored, total not updated	filledBasketReportsBothProperties	assertAll reports it alongside the size check rather than instead of it.
s06_parameterized	Off-by-one boundary: <code>>= 90</code> becomes <code>> 90</code>	gradeTable, on the row 90 → 1.0 only	23 of 24 parameterized cases still pass. A table without the exact boundary catches nothing.
s07_extras	Everything returns a constant zero	tokenIsAlwaysPositive, finishesQuickly, assertTimeoutOnOneCall	The timeout tests only catch it because they also assert the result .
s08_nest_order	remove() wipes the whole collection	removeDeletesOnlyThatTitle → assertTrue(contains("Solaris"))	Checking only that the removed item is gone passes . The survivor check is the one with teeth.
s10_collections	Prepend instead of append	orderIsPreserved → assertIterableEquals	Size and contains both pass. Only positional comparison sees it.

Scenario	Mutation applied	Caught by	Why nothing else catches it
s10_collections	remove() deletes every element	removeDeletesOnlyThatTrack → exact size + survivor check	—
s11_validation	Inverted boundary: < 0 becomes <= 0, so zero is wrongly rejected	zeroIsAccepted — and nothing else in the class	All five invalid-input tests still pass. This is the whole argument for testing the valid boundary.
s12_volidstate	registerMember() does nothing	registerMemberActuallyRegisters, registeringTwoMembersCountsBoth	⚠ Not caught by thisIsTheTestThatScoredZero, which is kept in the suite, labelled, as the weak shape never to submit.
s12_volidstate	Collection overwritten instead of appended	registeringTwoMembersCountsBoth → assertEquals(2, ...)	"The registry is not empty" passes. The exact count does not.
s13_calculations	VAT added instead of applied	normalGross → assertEquals(44.625, ...)	Caught only because the inputs are 12.50 × 3. With a net of 0.00 both formulas agree.
s13_calculations	Discount formula inverted (amount * rate)	partialDiscount, zeroDiscount, fullDiscount	A suite testing only rate 0.5 on an amount of 0.0 would catch nothing.
s14_query	Filter skipped, everything returned	filterKeepsOnlyAvailableBooks → exact size and assertFalse(contains(dune))	Checking only that the available book is present passes .
s15_observers	detach() does nothing	detachedViewIsNotNotified, detachRemovesOnlyThatView → assertEquals(0, updateCount)	Asserting a call did not happen is the only thing that catches a no-op removal.
s16_contracts	hashCode() no longer agrees with equals()	equalObjectsShareAHashCode, hashSetDeduplicates	Every equality test passes. A hash-based collection is what exposes the broken pair.
s17_boundaries	Off-by-one at the overdraft floor: < becomes <=	exactlyAtTheOverdraftLimit — and nothing else in the class	L−1 and L+1 both still pass. This single row is the case for boundary-value testing.
exam_library	registerMember() does nothing	testAddBooksAndRegisterMembers → assertEquals(3, getMemberCount())	⚠ The exact defect from the graded feedback.
exam_library	Pre-registered member missing	testInitialLibraryIsEmptyAndHasOneMember → assertEquals(1, getMemberCount())	⚠ A test that only checked getBookCount() sails straight past. This is the second lost point.
exam_library	Already-borrowed guard removed	testBorrowAlreadyBorrowedBookThrowsException	—
exam_library	removeBook() removes nothing	testRemoveBookAndMember → assertEquals(0, getBookCount()) after asserting it was 1	Asserting the before value is what makes the after value meaningful.
exam_cart	addProduct() does nothing	testAddProductsAndCalculatePrice, testRemoveProduct, testApplyDiscount	—
exam_cart	Total returns a constant zero	testAddProductsAndCalculatePrice (22.50), testApplyDiscount (20.25)	Not caught by testInitialCartIsEmpty — 0.0 is the right answer there. An empty-cart test can never prove a total is computed.

Scenario	Mutation applied	Caught by	Why nothing else catches it
exam_cart	Discount rate ignored	testApplyDiscount → assertEquals(20.25, ...)	Had the test used rate 0.0 or an empty cart, both versions agree and the mutant survives.
exam_cart	removeProduct() clears the whole cart	testRemoveProduct → count 1 and remaining total 2.50	The remaining total is what names <i>which</i> product survived.

Mock exam (Jun 2026) — the new exercises

Exercise	Mutation applied	Caught by	Why nothing else catches it
Mocking, SmartHome	register() never called on the device	testAddDevice → verify(deviceMock)	⚠ The list is still updated and true is still returned, so the state assertion AND the return-value assertion both stay green. register returns void — verify() is the only witness.
Mocking, SmartHome	duplicate check dropped	testAddDeviceTwiceIsRefused → unexpected second register(), size 1, return false	Caught three ways; a test without the second addDevice call catches none of them.
Mocking, SmartHome	removeDevice reports success without removing	testRemoveDevice → assertEquals(0, getDevices().size())	The return value is true in both versions. This is why the task says "attributes updated correctly ".
Mocking, SmartHome	sendCommand does nothing	testSendCommand → verify(deviceMock)	⚠ sendCommand returns void and changes no SmartHome state. There is literally no other observable difference.
Mocking, SmartHome	unregistered device is commanded anyway	testSendCommandToUnknownDevice → the unrecorded mock	The must-not-be-called pattern, in the new exam.
SOLID	PrerequisiteRule blocks courses with no prerequisite	noPrerequisiteDoesNotBlock — and nothing else	⚠ The "completed" and "missing" prerequisite tests both still pass. The null branch is the entire task-1 mark.
SOLID	only the student is updated, not the course	successUpdatesBothSides → the course-side assertions	Checking only the student passes, and the course then still reports a free seat it has given away.
SOLID	state mutated before the rules are evaluated	failureModifiesNothing → the state assertions after the refusal	The return value is false either way.
SOLID	duplicate enrollment check missing	duplicateIsRejected → sizes of 1 on both sides	Both return values still look plausible; only the counts expose it.
Black box	off-by-one at the VIP lower limit (>= → >)	TC11 (level 10) — and nothing else	⚠ TC10 (9) and TC12 (11) both pass. The value exactly <i>on</i> an inclusive boundary is the entire point of the three-point rule.
Acceptance	invalid credentials are accepted	the second Gherkin scenario → assertFalse(isAccessGranted())	A happy-path-only scenario set never notices. Hence "two scenarios".

ISE HN 2026 endterm (Aug 2026) — CloudServer and Graphics

Exercise	Mutation applied	Caught by	Why nothing else catches it
exam_clo udserver	freeRam wraps with % instead of capping (the bug in the code you submitted)	testFreeRamCap → assertEquals(680.0, ...)	⚠️ testFreeRamNormal passes — 23 is below the cap, so % and min agree. Only a value above the cap tells them apart.
exam_clo udserver	the +40 speed bonus is never added	testSuccessfulAllocationSameTier → the speed assertion	isSuccess(), the cost and the RAM left are all unchanged, so only the speed line notices.
exam_clo udserver	same-tier discount ignored (always 1.5 × load)	testSuccessfulAllocationSameTier → cost 3.0 and RAM 47.0	The different-tier test uses 1.5 anyway — it cannot see this.
exam_clo udserver	ram < cost became ram <= cost	testAllocationWithExactlyEnoughRamSucceeds — and nothing else	⚠️ The insufficient-RAM test (10 vs 15) fails the same way in both versions. The exact-fit case is the only one that separates < from <=.
Black box, exam_graphics	baseline exclusive: 2048 → unsupported	TC11 (Mobile, 2048) — and nothing else	⚠️ TC10 (2047) and TC12 (2049) both pass. Same lesson as the game launcher: the marks are on the value <i>at</i> the limit.

Did any mutation escape?

No. All 51 were caught on the first run, so no test needed strengthening after the fact. Three tests were, however, **written deliberately** to survive a mutation and are kept in the suite as labelled counter-examples:

Test	Survives	Kept because
s12_voidstate.thisIsTheTestThatScoredZero	registerMember() doing nothing	It is the exact shape of the test that earned the graded feedback. Seeing it stay green next to the tests that go red is the lesson.
s14_mustnotbecalled.niceMockHidesTheBug	The forbidden collaborator call	Shows concretely why @Mock(type = MockType.NICE) scores zero on a must-not-be-called task.
s12_callorder.wrongOrderSlipsPastADefaultMock	The unsafe call order	Shows why order requirements need createStrictMock.

These three assert `assertDoesNotThrow(...)` on purpose, so they are green *and* honest about what they cannot see.

Scenarios with no mutant, and why

Scenario	Reason
mocking.s04_mocktypes	Already proves its own failures inline with <code>assertThrows(AssertionError.class, ...)</code> against real broken call sequences. Adding an external mutant would be circular.
mocking.s09_matchers	Matcher syntax, not SUT behaviour. <code>MixedMatcherErrorTest</code> proves the illegal form throws and the <code>eq()</code> fix works.
mocking.s11_dynamicretur ns	The values are computed from the arguments , so the test already fails on any SUT that ignores the collaborator.

Scenario	Reason
mocking.s12_callorder	Ships <code>UnsafeAirlockController</code> as a real class in <code>src/main</code> and asserts the strict mock rejects it — the mutant is the scenario.
mocking.s14_mustnotbecalled	Same: <code>CarelessThesisOffice</code> is the mutant, and every technique is run against it in the scenario itself.
mocking.s15_partialmocks	The mocked seam is the injected fault; both branches of <code>isYearEnd()</code> are asserted.
mocking.s16_injectionseams	Teaches where a mock can be inserted. <code>UnmockableCoffeeMachine</code> is itself the counter-example.
mocking.s17_errorcatalogue	Every test is <code>@Disabled</code> and exists to be run for its error message .
testing.s01_lifecycle	Asserts JUnit's own behaviour (hook order, fresh instance per test), not a SUT.
testing.s08_nested_ordered.OrderedWorkflowTest	<code>PER_CLASS</code> shared-state demo; the mutant lives on <code>CatalogueStructureTest</code> , which covers the same SUT.
testing.s09_assumptions	Asserts the environment. A mutant would only prove that <code>assumeTrue</code> skips, which the scenario already shows both ways.

The pre-submission checklist

Derived from the table above. Run through it on every graded test before you commit.

1. **Did I assert the return value or the resulting state** — not merely that nothing threw?
2. **Is it an exact number?** `assertEquals(2, ...)`, not `assertFalse(isEmpty())`.
3. **Is there a negative assertion?** The removed item is gone; the filtered item is absent; the detached observer got 0 updates.
4. **Would it fail if the method body were deleted?** For a void method, the answer is only yes if you asserted state — or, with mocks, called `verify()`.
5. **Would it fail if the operation were inverted?** Add instead of remove, `>` instead of `>=`, an empty list instead of a filtered one.
6. **Did I test the valid boundary too?** `L-1`, `L`, `L+1` — not just the invalid side.
7. **For doubles:** a delta, and inputs where a wrong formula gives a *different* number. Never 0.0 or 1.0 alone.
8. **For mocks:** `verify()` **and** an assertion. And for "must not be called": a **default** mock with the call left unrecorded.